

Ex 1: Control Structures(PL/SQL)

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
DECLARE
```

```
    CURSOR customer_cursor IS
```

```
        SELECT c.CustomerID, l.LoanID, l.InterestRate
```

```
        FROM Customers c
```

```
        JOIN Loans l ON c.CustomerID = l.CustomerID
```

```
        WHERE EXTRACT(YEAR FROM SYSDATE) - EXTRACT(YEAR FROM c.DOB) > 60;
```

```
BEGIN
```

```
    FOR loan_record IN customer_cursor LOOP
```

```
        UPDATE Loans
```

```
        SET InterestRate = InterestRate - 1
```

```
        WHERE LoanID = loan_record.LoanID;
```

```
        DBMS_OUTPUT.PUT_LINE('Applied 1% discount to loan ID: ' || loan_record.LoanID);
```

```
    END LOOP;
```

```
END;
```

Scenario 2: A customer can be promoted to VIP status based on their balance.

Question: Write a PL/SQL block that iterates through all customers and sets a flag *IsVIP* to *TRUE* for those with a balance over \$10,000.

```
ALTER TABLE Customers ADD (IsVIP CHAR(1));
```

```
DECLARE
```

```
    CURSOR customer_cursor IS
```

```
        SELECT CustomerID, Balance
```

```
        FROM Customers;
```

```
BEGIN
```

```
    FOR customer_record IN customer_cursor LOOP
```

```
        IF customer_record.Balance > 10000 THEN
```

```
            UPDATE Customers
```

```
            SET IsVIP = 'Y'
```

```
            WHERE CustomerID = customer_record.CustomerID;
```

```
            DBMS_OUTPUT.PUT_LINE('Promoted to VIP status for customer ID: ' ||  
customer_record.CustomerID);
```

```
        ELSE
```

```
            UPDATE Customers
```

```
            SET IsVIP = 'N'
```

```
            WHERE CustomerID = customer_record.CustomerID;
```

```
        END IF;
```

```
    END LOOP;
```

```
END;
```

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
DECLARE
```

```
    CURSOR loan_cursor IS
```

```
        SELECT l.LoanID, l.CustomerID, l.EndDate, c.Name
```

```
        FROM Loans l
```

```
        JOIN Customers c ON l.CustomerID = c.CustomerID
```

```
        WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30;
```

```
BEGIN
```

```
    FOR loan_record IN loan_cursor LOOP
```

```
        DBMS_OUTPUT.PUT_LINE('Reminder: Loan ID ' || loan_record.LoanID ||
```

```
                               ' for customer ' || loan_record.Name ||
```

```
                               ' is due on ' || loan_record.EndDate);
```

```
    END LOOP;
```

```
END;
```

Exercise 3: Stored Procedures(PL/SQL)

Scenario 1: *The bank needs to process monthly interest for all savings accounts.*

Question: *Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.*

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance * 1.01
    WHERE AccountType = 'Savings';

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Monthly interest applied to all savings accounts.');
```

END ProcessMonthlyInterest;

Scenario 2: *The bank wants to implement a bonus scheme for employees based on their performance.*

Question: *Write a stored procedure UpdateEmployeeBonus that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.*

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_department IN VARCHAR2,
```

```

    p_bonus_percentage IN NUMBER
) AS
BEGIN
    UPDATE Employees
    SET Salary = Salary * (1 + p_bonus_percentage / 100)
    WHERE Department = p_department;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Bonus applied to all employees in department: ' ||
p_department);
END UpdateEmployeeBonus;

```

Scenario 3: *Customers should be able to transfer funds between their accounts.*

Question: *Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.*

```

CREATE OR REPLACE PROCEDURE TransferFunds (
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
) AS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance

```

FROM Accounts

WHERE AccountID = p_from_account;

IF v_balance < p_amount THEN

RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds in account ' || p_from_account);

END IF;

BEGIN

UPDATE Accounts

SET Balance = Balance - p_amount

WHERE AccountID = p_from_account;

UPDATE Accounts

SET Balance = Balance + p_amount

WHERE AccountID = p_to_account;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Transfer of ' || p_amount || ' from account ' ||
p_from_account || ' to account ' || p_to_account || ' completed successfully.');

EXCEPTION

WHEN OTHERS THEN

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

END TransferFunds;